

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência

Inteligência Artificial e Ilusão de Inteligência

Semana 2 — Máquinas de Estado Finitas

Módulo 1: Como um NPC decide o que fazer?

Apostila: Parte II, Cap. 3

Micro Game: NPC Decision

IFMS • Semana 02

Objetivos da aula

Ao final de hoje você será capaz de:

- explicar por que condicionais soltos não bastam para decidir;
- definir estado, transição, evento e ação;
- aplicar o ciclo enter/update/exit;
- distinguir FSM por polling de FSM por eventos;
- implementar uma primeira FSM no Micro Game NPC Decision.

Retomada da Semana 1

O ciclo **Sentir** → **Pensar** → **Agir** é o modelo de todo agente.

Hoje aprofundamos a etapa **Pensar**: como organizar a decisão do NPC ao longo do tempo.

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência



Como um NPC decide o que fazer?

Um guarda ouve um ruído, investiga, e depois volta a patrulhar. Como isso é "lembrado"?

IFMS • Semana 02

O problema dos condicionais soltos

Uma cadeia de `if/else` reavaliada a cada quadro **não tem memória de contexto**.

⚠️ ATENÇÃO

O guarda esquece que estava investigando assim que o ruído para — e volta a patrulhar cedo demais.

A resposta: Máquina de Estados Finita

Um NPC está sempre em **um único estado**, e só muda de estado por uma transição explícita.

- Isso **é** a memória de contexto que faltava
- Base teórica: **autômatos finitos** — finitos e determinísticos

DICA

Por ser finita e determinística, a FSM pode ter **todos os seus estados enumerados** e **todas as suas transições revisadas antecipadamente**. É essa propriedade — não acaso — que torna a FSM **previsível e depurável**: dá para responder sempre "por que o NPC está fazendo isso agora?".

Os quatro conceitos elementares

Conceito	O que representa
Estado	Um modo de comportamento (ex.: Patrulhar)
Transição	Mudança de um estado para outro
Evento / Guarda	Condição que dispara a transição
Ação	O que o NPC faz dentro do estado

Exemplo: três estados



IFMS • Semana 02

Guardas de transição e prioridade

Duas guardas podem ser verdadeiras ao mesmo tempo.



DICA

"Distância < 2m" e "vida < 10%" — qual vence? A prioridade precisa ser definida conscientemente, não por acidente de ordem no código.

Funcionamento: polling vs. eventos

- **Polling** — a condição é checada a cada quadro
- **Eventos** — a transição é disparada apenas quando algo muda

NA INDÚSTRIA

Polling é mais simples de implementar; eventos custam menos processamento quando a condição raramente muda.

O ciclo enter / update / exit

Cada estado tem três momentos distintos.

Momento	Quando ocorre	Exemplo
Enter	Uma vez, ao entrar	Tocar som de alerta
Update	A cada quadro, enquanto permanece	Mover em direção ao alvo
Exit	Uma vez, ao sair	Parar animação de perseguição

⚠️ ATENÇÃO

Erro comum: colocar o som de alerta no *update* — ele toca a cada quadro, não uma vez só.

Exemplo canônico: o guarda de 5 estados

Patrulhar → Investigar → Perseguir → Atacar → Fugir

Cada seta é uma transição guardada por uma condição específica.

Aplicações em jogos comerciais

- **Pac-Man (1980)** — fantasmas com estados *chase, scatter, frightened*
- **Half-Life (1998)** — esquadrões de soldados coordenados por FSM

NA INDÚSTRIA

A FSM continua presente mesmo em jogos modernos, geralmente como camada de baixo nível dentro de estruturas maiores.

Por que a FSM é tão usada

- **Simples** — mapeia o modo como já descrevemos comportamento
- **Barata** — viável para centenas de agentes
- **Previsível e depurável** — consequência direta de ser finita e determinística
- **Visualizável** — se presta a editores gráficos, como o Animator a seguir

FSM no Unity: Animator Controller

O Animator materializa visualmente os conceitos estudados.

Conceito teórico	Elemento no Animator
Estado	Nó do grafo
Transição	Seta entre nós
Guarda	Parâmetro (bool, float, trigger)

DICA

O pacote **Visual Scripting** oferece grafos equivalentes para montar a mesma lógica sem escrever código — mesma ideia, outra ferramenta.

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência

Implementação guiada

Construindo a primeira versão do Micro Game NPC Decision.

IFMS • Semana 02

Do esboço à FSM

Retome o esboço da Semana 1 e defina, em grupo:

1. de três a cinco estados;
2. as transições entre eles;
3. a prioridade entre guardas concorrentes.

Implementação no Unity

Três caminhos possíveis, conforme o domínio do grupo:

- **Animator Controller** — mais visual
- **Script com** `enum` / `switch` — mais direto
- **Padrão de projeto** *State* — mais extensível

DICA

O conceito avaliado é o mesmo nas três abordagens.



Erro comum

Tentar implementar já na primeira versão um número excessivo de estados.

OBJETIVOS

Mantenha o escopo entre três e cinco estados — o suficiente para observar o comportamento sem antecipar a explosão de transições.

Discussão técnica: os limites da FSM plana

E se adicionarmos "Esconder-se" e "Chamar reforços" ao guarda?

⚠️ ATENÇÃO

O número de transições possíveis cresce de forma quadrática com o número de estados — a explosão de transições.

Resumo da semana

- FSM resolve a falta de memória de contexto dos condicionais soltos
- Estado, transição, evento e ação são os quatro conceitos elementares
- O ciclo enter/update/exit evita ações repetidas por engano
- O Animator Controller materializa a FSM visualmente na Unity
- Uma FSM plana com muitos estados sofre explosão de transições

Preparação para a Semana 3

Tema: Aprofundamento da FSM — o padrão de projeto State em C#

- Não há capítulo novo: revisar as seções 3.2, 3.3.2 e 3.7 da Apostila (já lidas para hoje)
- Trazer a FSM implementada hoje, funcionando no Micro Game

DICA

Pergunta que abre a Semana 3: não é "como organizar hierarquicamente" — é "como estruturar o **código** de uma FSM que já apresenta sinais de explosão de transições?"